Лабораторна робота №3

Розв'язання комбінаторної задачі розміщення з обмеженнями з використанням генетичного алгоритму та алгоритму імітації відпалу

Мета: Вивчення принципів роботи генетичних алгоритмів, набуття навичок самостійного кодування розв'язку у вигляді хромосоми та побудови фітнес-функції для задачі з обмеженнями, а також порівняння ефективності генетичного алгоритму й алгоритму імітації відпалу на одній і тій самій задачі.

Варіант №3

Постановка задачі. Розмістити на квадратному полі з рифами флот кораблів гри «Морський бій» так, аби кожен корабель був горизонтальним або вертикальним, лежав у межах поля, не стояв на рифі, не перетинався з іншими кораблями й не торкався їх ні стороною, ні кутом. Вміст JSON-файлу варіанта:

```
{"field_size": 8, "ships": [3, 2, 2, 2, 1, 1, 1, 1, 1, 1], "reefs": [[0, 1], [0, 3],
[0, 7], [1, 1], [1, 4], [1, 6], [1, 7], [2, 0], [2, 1], [2, 5], [2, 6], [2, 7], [3,
2], [3, 3], [3, 5], [3, 6], [4, 1], [4, 2], [4, 3], [4, 5], [4, 6], [5, 0], [5, 2],
[5, 3], [5, 4], [5, 5], [5, 6], [6, 0], [6, 1], [6, 4], [6, 6], [7, 0], [7, 1], [7,
3], [7, 4]]}
```

Завдання 1. Реалізувати програму, яка зчитує варіант із JSON-файлу і перевіряє коректність вхідних даних (рифи в межах поля, кораблі не довші за поле).

```
def load_variant(path):
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    n = data["field_size"]
    ships = list(data["ships"])
    reefs = {(int(x), int(y)) for x, y in data["reefs"]}

    if not isinstance(n, int) or n <= 0:
        raise ValueError("field_size має бути натуральним числом")
    for x, y in reefs:
        if not (0 <= x < n and 0 <= y < n):
            raise ValueError(f"Риф ({x}, {y}) за межами поля {n}x{n}")
    for length in ships:
        if length < 1 or length > n:
            raise ValueError(f"Корабель довжиною {length} не вміщується в поле {n}x{n}")
    if sum(ships) > n * n - len(reefs):
        raise ValueError("Кораблі не вміщуються у вільні клітинки поля")
    return n, ships, reefs
```

Завдання 2. Самостійно розробити кодування розстановки у вигляді хромосоми. Ген – це номер розміщення для окремого корабля, котрий допустимий. Для кожного судна формується список з усіх можливих варіантів розташування за координатами, котрі розміщують в межах ігрового поля та оминають рифи. Довжина хромосоми – це 10 генів по одному на кожен корабель. Відповідно, будь-який ген є цілим числом від 0 до загального розміру списку цього номеру корабля (до прикладу: у моєму варіанті це [8, 23, 23, 23, 29, 29, 29, 29, 29, 29]). Декодування працює як розміщення якогось корабля і визначається як domains[i][gen[i]]. То враховуючи це значення та довжину корабля вже вираховуються всі клітинки, котрі він може зайняти. Тому таке кодування також дозволяє автоматично дотримуватися правил щодо орієнтації, межі поля та відсутністю кораблів на рифах, оскільки неприпустимі розміщення від самого початку відкинуті й відсутні в списках. Проте правила перетину кораблів і дотику не рахуються автоматично, тож аби їх контролювати, ці умови враховує фітнес-функція.

```
def __init__(self, n, ships, reefs):
    self.n, self.ships, self.reefs = n, ships, reefs
    self.evals = 0
    self.domains = [self._placements(length) for length in ships]
    for i, d in enumerate(self.domains):
        if not d:
            raise ValueError(f"Для корабля №{i + 1} немає жодного місця на полі")
    self.near = [self._neighbours(d) for d in self.domains]

def _placements(self, length):
    res = []
    for o in (HORIZONTAL, VERTICAL):
        if length == 1 and o == VERTICAL:
            continue
        for y in range(self.n):
            for x in range(self.n):
                cs = self.cells(length, (x, y, o))
                if all(0 <= cx < self.n and 0 <= cy < self.n
                       and (cx, cy) not in self.reefs for cx, cy in cs):
                    res.append((x, y, o))
    return res

def random_chromosome(self):
    return [random.randrange(len(d)) for d in self.domains]

def decode(self, chrom):
    return [self.domains[i][g] for i, g in enumerate(chrom)]

@staticmethod
def cells(length, placement):
    x, y, o = placement
    if o == HORIZONTAL:
        return [(x + k, y) for k in range(length)]
    return [(x, y + k) for k in range(length)]
```

Завдання 3. Самостійно розробити фітнес-функцію.

Фітнес-функцію побудовано як суму штрафів за кожне порушення, котра береться зі знаком мінус: $F(x) = -(w_2*P_2 + w_3*P_3 + w_4*P_4 + w_5*P_5)$.

Де P_2 рахує клітинки кораблів, котрі вилізли за межі поля, а P_3 котрі потрапили на рифи. За ці порушення штраф найбільший - по 5 балів. P_4 – це кількість клітинок, де кораблі накладаються один на одного (себто це перетин, а вага 3), тоді як P_5 кількість дотиків між різними кораблями по стороні чи діагоналі (вага 1). Перше правило щодо орієнтації у формулі відсутнє, оскільки воно вже автоматично виконується на етапі кодування хромосоми.

Аби алгоритм міг відрізнити дві неправильні розстановки й обрати кращу, штрафи рахуються в клітинках. Себто, розстановка лише з одним дотиком отримає фітнес −1. Попри це варіант з одним перетином і трьома дотиками дасть у сумі −6. Оскільки −1 більше за −6, алгоритм бачить, що перший розрахунок ближчий до правильної, то і надає їй перевагу під час відбору.

Отже, коли розстановка повністю правильна - то штрафів немає взагалі. Тому ідеальному рішенню відповідає фітнес, котрий дорівнює 0. Тож якщо дійде до нульового значення - це використовується як критерій для зупинки роботи алгоритму.

```
W_BOUNDS, W_REEF, W_OVERLAP, W_TOUCH = 5, 5, 3, 1

def analyse(self, placements, collect=False):
    n = self.n
    owner, all_cells = {}, []
    out = reef = 0
    bad = set()
    for i, (length, p) in enumerate(zip(self.ships, placements)):
        cs = self.cells(length, p)
        all_cells.append(cs)
        for c in cs:
            if not (0 <= c[0] < n and 0 <= c[1] < n):
                out += 1
                bad.add(c)
            elif c in self.reefs:
                reef += 1
                bad.add(c)
            owner.setdefault(c, []).append(i)
    overlap = sum(len(v) - 1 for v in owner.values())
    touch = 0
    for i, cs in enumerate(all_cells):
        for (x, y) in cs:
            for dx in (-1, 0, 1):
                for dy in (-1, 0, 1):
                    if dx == 0 and dy == 0:
                        continue
                    for j in owner.get((x + dx, y + dy), ()):
                        if j > i:
                            touch += 1
```

```
                            if collect:
                                bad.add((x, y))
                                bad.add((x + dx, y + dy))
    if collect:
        bad.update(c for c, v in owner.items() if len(v) > 1)
    return {"bounds": out, "reef": reef, "overlap": overlap, "touch": touch}, bad

def penalty(self, v):
    return (W_BOUNDS * v["bounds"] + W_REEF * v["reef"]
            + W_OVERLAP * v["overlap"] + W_TOUCH * v["touch"])

def fitness(self, chrom):
    self.evals += 1
    v, _ = self.analyse(self.decode(chrom))
    return -self.penalty(v)
```

Завдання 4. Реалізувати генетичні оператори: відбір, схрещування, мутацію. Оператори схрещування й мутації мають відповідати вибраному кодуванню.

Параметр	Значення	Призначення
POP_SIZE (M)	100	розмір популяції
P_CROSS (p_c)	0.9	ймовірність схрещування
P_MUT (p_m)	0.1	ймовірність мутації гена
TOURNAMENT_K	3	розмір турніру
ELITE	2	кількість елітних особин
MAX_GEN	300	максимум поколінь

```
def tournament(pop, fits, k=TOURNAMENT_K):
    idx = random.sample(range(len(pop)), k)
    return pop[max(idx, key=lambda i: fits[i])]

def crossover(p1, p2):
    L = len(p1)
    a, b = sorted(random.sample(range(L + 1), 2))
    return p1[:a] + p2[a:b] + p1[b:], p2[:a] + p1[a:b] + p2[b:]

def mutate(problem, chrom, pm):
    res = chrom[:]
    for i in range(len(res)):
        if random.random() < pm:
            if random.random() < 0.5:
                res[i] = random.randrange(len(problem.domains[i]))
            else:
                res[i] = problem.shift_gene(i, res[i])
    return res

def shift_gene(self, i, g):
    nb = self.near[i][g]
    return random.choice(nb) if nb else random.randrange(len(self.domains[i]))

def run_ga(problem, pop_size=POP_SIZE, pc=P_CROSS, pm=P_MUT, k=TOURNAMENT_K,
           elite=ELITE, max_gen=MAX_GEN):
    t0 = time.time()
    problem.evals = 0
    pop = [problem.random_chromosome() for _ in range(pop_size)]
    fits = [problem.fitness(c) for c in pop]
    best_hist, mean_hist = [], []
    solved_evals = None
```

```
        for gen in range(max_gen + 1):
            best_i = max(range(pop_size), key=lambda i: fits[i])
            best_hist.append(fits[best_i])
            mean_hist.append(sum(fits) / pop_size)

            if fits[best_i] == 0:
                solved_evals = problem.evals
                break

            if gen == max_gen:
                break
            order = sorted(range(pop_size), key=lambda i: fits[i], reverse=True)
            new_pop = [pop[i][:] for i in order[:elite]]
            new_fits = [fits[i] for i in order[:elite]]
            while len(new_pop) < pop_size:
                p1, p2 = tournament(pop, fits, k), tournament(pop, fits, k)
                c1, c2 = crossover(p1, p2) if random.random() < pc else (p1[:], p2[:])
                for c in (mutate(problem, c1, pm), mutate(problem, c2, pm)):
                    if len(new_pop) < pop_size:
                        new_pop.append(c)
                        new_fits.append(problem.fitness(c))
            pop, fits = new_pop, new_fits

    best_i = max(range(pop_size), key=lambda i: fits[i])
    return {"chrom": pop[best_i], "fitness": fits[best_i], "success": fits[best_i] == 0,
            "generations": len(best_hist) - 1, "evals": solved_evals or problem.evals,
            "time": time.time() - t0, "best_hist": best_hist, "mean_hist": mean_hist}
```

Завдання 4. Реалізувати візуалізацію: поле з рифами та знайденою розстановкою кораблів (кораблі мають бути про нумеровані, клітинки з порушеннями правил – виділені кольором); графік найкращого та середнього значення фітнес-функції за поколіннями.

```
def draw_board(problem, chrom, ax, title=""):
    n = problem.n
    pl = problem.decode(chrom)
    _, bad = problem.analyse(pl, collect=True)
    ax.set_xlim(0, n)
    ax.set_ylim(n, 0)
    ax.set_aspect("equal")
    for y in range(n):
        for x in range(n):
            color = "#dff1fb"
            if (x, y) in problem.reefs:
                color = "#6b7a8f"
        ax.add_patch(Rectangle((x, y), 1, 1, facecolor=color, edgecolor="#9fb6c8", lw=0.8))
    palette = ["#aec7e8", "#ffbb78", "#98df8a", "#c5b0d5", "#dbdb8d",
               "#9edae5", "#c7c7c7", "#f7b6d2", "#c49c94", "#8fd3c1"]
    for i, (length, p) in enumerate(zip(problem.ships, pl)):
        for (x, y) in problem.cells(length, p):
            if not (0 <= x < n and 0 <= y < n):
                continue
            color = "#e53935" if (x, y) in bad else palette[i % len(palette)]
          ax.add_patch(Rectangle((x, y), 1, 1, facecolor=color, edgecolor="black", lw=1.4))
            ax.text(x + 0.5, y + 0.5, str(i + 1), ha="center", va="center",
                    fontsize=11, fontweight="bold")
    ax.set_xticks([k + 0.5 for k in range(n)])
    ax.set_xticklabels(range(n))
    ax.set_yticks([k + 0.5 for k in range(n)])
    ax.set_yticklabels(range(n))
    ax.xaxis.tick_top()
    ax.tick_params(length=0)
    ax.set_title(title, pad=24)
    ax.legend(handles=[Patch(facecolor="#6b7a8f", label="риф"),
                       Patch(facecolor="#e53935", label="порушення")],
              loc="upper center", bbox_to_anchor=(0.5, -0.02), ncol=2, frameon=False)
```

```python
def show_ga(problem, res, out="ga_result.png"):
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(12, 5.5))
    draw_board(problem, res["chrom"], a1,
               f"ГА: F = {res['fitness']}, покоління {res['generations']}")
    a2.plot(res["best_hist"], label="найкраща", color="tab:green")
    a2.plot(res["mean_hist"], label="середня", color="tab:blue")
    a2.xaxis.set_major_locator(matplotlib.ticker.MaxNLocator(integer=True))
    a2.set_xlabel("покоління")
    a2.set_ylabel("фітнес F")
    a2.grid(alpha=0.3)
    a2.legend()
    fig.tight_layout()
    fig.savefig(out, dpi=130)
    plt.show()

def print_solution(problem, chrom, fitness):
    pl = problem.decode(chrom)
    for i, (length, (x, y, o)) in enumerate(zip(problem.ships, pl), 1):
        print(f"Корабель №{i:2d}: довжина {length}, перша клітинка ({x}, {y}),
{ORIENT_NAME[o]}")
    print("Значення фітнес-функції:", fitness)
    v, _ = problem.analyse(pl)
    if fitness == 0:
        print("Розстановка правильна, порушень немає.")
    else:
        print("Правильну розстановку не знайдено. Порушені правила:")
        for key, name in RULES.items():
            if v[key]:
                print(f"  - {name}: {v[key]}")
```

```
E:\обчисл\lab3\lab3_bubon\.venv\Scripts\python.exe E:\обчисл\lab3\lab3_bubon\main.py
Бубон Л. В., варіант 3; поле 8x8, кораблів: 10, рифів: 35
Корабель № 1: довжина 3, перша клітинка (2, 4), горизонтально
Корабель № 2: довжина 2, перша клітинка (6, 7), горизонтально
Корабель № 3: довжина 2, перша клітинка (3, 7), горизонтально
Корабель № 4: довжина 2, перша клітинка (0, 0), горизонтально
Корабель № 5: довжина 1, перша клітинка (6, 5), горизонтально
Корабель № 6: довжина 1, перша клітинка (5, 1), горизонтально
Корабель № 7: довжина 1, перша клітинка (0, 6), горизонтально
Корабель № 8: довжина 1, перша клітинка (0, 2), горизонтально
Корабель № 9: довжина 1, перша клітинка (3, 0), горизонтально
Корабель №10: довжина 1, перша клітинка (6, 3), горизонтально
Значення фітнес-функції: 0
Розстановка правильна, порушень немає.
Поколінь: 7, обчислень F: 786, час: 0.03 с
```

Рис. 1. Результат виконання.

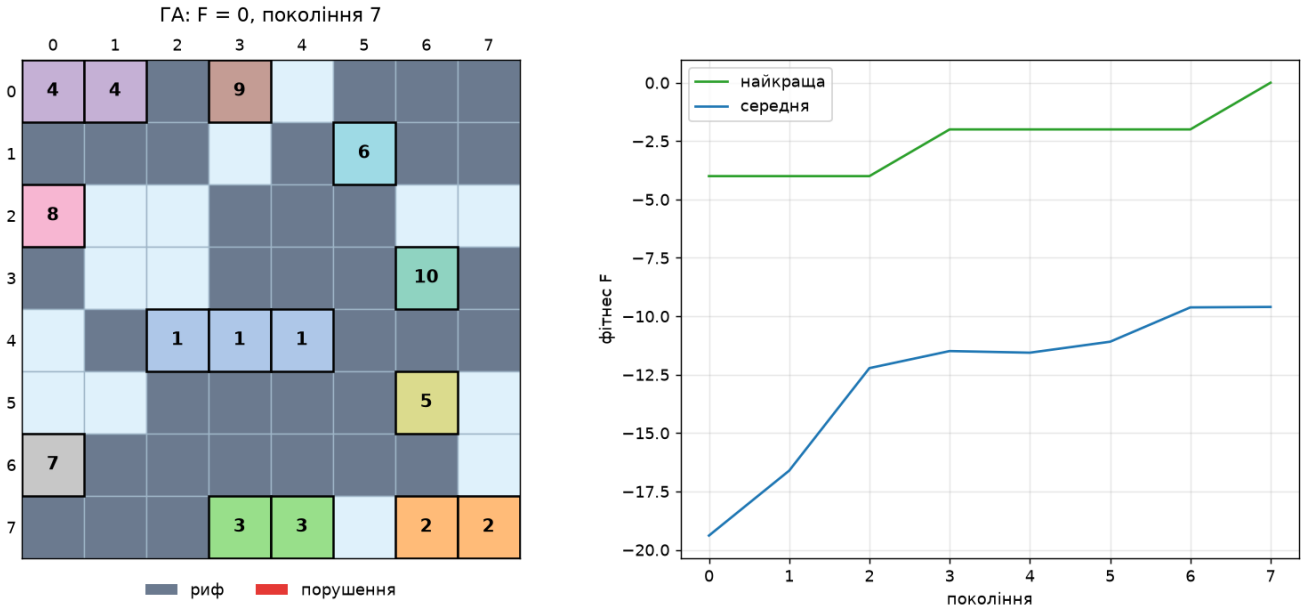


Рис. 2. Знайдена розстановка та графік фітнесу за поколіннями.

Завдання 6. Провести дослідження впливу параметрів ГА на результат. Для кожного з параметрів – розмір популяції M та ймовірність мутації p_m – розглянути щонайменше три значення і для кожного значення виконати щонайменше 5 запусків. Зафіксувати частку успішних запусків і середню кількість поколінь до знаходження розв'язку.

Вплив розміру популяції M ($p_m = 0.1$):

Значення	Успішних запусків	Середня к-сть ітерацій	Середнє обчислень F	Середній час, с
10	90.0%	35.4	293	0.01
20	100.0%	28.3	530	0.01
50	100.0%	25.9	1293	0.04
100	100.0%	16.4	1704	0.04
200	100.0%	13.3	2840	0.07

Вплив ймовірності мутації p_m (M = 100):

Значення	Успішних запусків	Середня к-сть ітерацій	Середнє обчислень F	Середній час, с
0.01	96.7%	18.2	1888	0.08
0.05	100.0%	11.7	1243	0.03
0.1	100.0%	17.1	1773	0.05
0.3	93.3%	87.5	8675	0.29
0.6	20.0%	197.5	19455	0.83

Завдання 7. Реалізувати розв'язання тієї самої задачі алгоритмом імітації відпалу (SA). Програма має працювати з тими самими JSON-файлами варіантів, що й у завданні 1.

Поточний стан - це та ж хромосома, що й у попередньому завданні, тому всі старі файли підходять. Функція енергії E - це фітнес-функція з минулого завдання, тільки без мінуса. І тут теж мета дійти до нуля, оскільки, як вже я описувала, нуль означає повністю правильну розстановку.

Новий сусідній стан створюється тим, що береться рандомно один корабель. З ймовірністю 70% відбувається зсуа на одну клітинку або повертається. У решті випадків (себто з ймовірністю 30%) - закидається на нове рандомне місце, котре є в його списку ймовірних позицій.

Початкова температура дорівнює 3 - це потрібно, аби на початку алгоритм міг робити рандомні стрибки, приймати гірші кроки й аби не застрягав на одному місці. Мінімальна температура тут 0.05, адже за такого алгоритм уже відкидає будь-які погіршення. Охолодження систему плавно множиться температуру на 0.95, роблячи по 150 спроб на кожному рівні.

Зупиняє програму тоді, коли E = 0 або коли температура впала нижче за мінімум.

```
def energy(problem, state):
    return -problem.fitness(state)

def neighbor_state(problem, state):
    new = state[:]
    i = random.randrange(len(new))
    if random.random() < P_LOCAL:
        new[i] = problem.shift_gene(i, new[i])
    else:
        new[i] = random.randrange(len(problem.domains[i]))
    return new

def run_sa(problem, t_start=T_START, t_min=T_MIN, alpha=ALPHA, chain=CHAIN_LEN):
    t0 = time.time()
    problem.evals = 0
    cur = problem.random_chromosome()
    e_cur = energy(problem, cur)
    best, e_best = cur[:], e_cur
    hist = {"T": [], "cur": [], "best": [], "acc": []}
    solved_evals = None
    T = t_start
    while T > t_min and e_best > 0:
        accepted = 0

        for _ in range(chain):
            cand = neighbor_state(problem, cur)
            e_cand = energy(problem, cand)
            delta = e_cand - e_cur
            if delta <= 0 or random.random() < math.exp(-delta / T):
                cur, e_cur = cand, e_cand
                accepted += 1
```

```
                if e_cur < e_best:
                    best, e_best = cur[:], e_cur
                    if e_best == 0:
                        solved_evals = problem.evals
                        break
        hist["T"].append(T)
        hist["cur"].append(e_cur)
        hist["best"].append(e_best)
        hist["acc"].append(accepted / chain)
        T *= alpha
    return {"chrom": best, "fitness": -e_best, "success": e_best == 0,
            "steps": len(hist["T"]), "evals": solved_evals or problem.evals,
            "time": time.time() - t0, "hist": hist}
```

Завдання 8. Реалізувати візуалізацію: поле з рифами та знайденою розстановкою кораблів (аналогічно до завдання 1); графіки зміни поточної та найкращої енергії, температури T і частки прийнятих переходів залежно від номера ступеня температури (або ітерації).

```
def show_sa(problem, res, out="sa_result.png"):
    h = res["hist"]
    fig = plt.figure(figsize=(13, 8))
    ax = fig.add_subplot(1, 2, 1)
    draw_board(problem, res["chrom"], ax, f"SA: F = {res['fitness']}, ступенів T:
{res['steps']}")
    ax1 = fig.add_subplot(3, 2, 2)
    ax1.plot(h["cur"], label="поточна E")
    ax1.plot(h["best"], label="найкраща E")
    ax1.set_ylabel("E")
    ax1.legend()
    ax2 = fig.add_subplot(3, 2, 4)
    ax2.plot(h["T"], color="tab:red")
    ax2.set_ylabel("T")
    ax3 = fig.add_subplot(3, 2, 6)
    ax3.plot(h["acc"], color="tab:purple")
    ax3.set_ylabel("частка прийнятих")
    ax3.set_xlabel("номер ступеня температури")
    for a in (ax1, ax2, ax3):
        a.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(out, dpi=130)
    plt.show()
```

```
Бубон Л. В., варіант 3; поле 8x8, кораблів: 10, рифів: 35
Корабель № 1: довжина 3, перша клітинка (7, 5), вертикально
Корабель № 2: довжина 2, перша клітинка (6, 2), горизонтально
Корабель № 3: довжина 2, перша клітинка (3, 0), горизонтально
Корабель № 4: довжина 2, перша клітинка (3, 4), горизонтально
Корабель № 5: довжина 1, перша клітинка (0, 2), горизонтально
Корабель № 6: довжина 1, перша клітинка (3, 7), горизонтально
Корабель № 7: довжина 1, перша клітинка (0, 4), горизонтально
Корабель № 8: довжина 1, перша клітинка (2, 2), горизонтально
Корабель № 9: довжина 1, перша клітинка (0, 0), горизонтально
Корабель №10: довжина 1, перша клітинка (0, 6), горизонтально
Значення фітнес-функції: 0
Розстановка правильна, порушень немає.
Ступенів T: 25, обчислень E: 3649, час: 0.10 с
```

Рис. 3. Результат виконання.

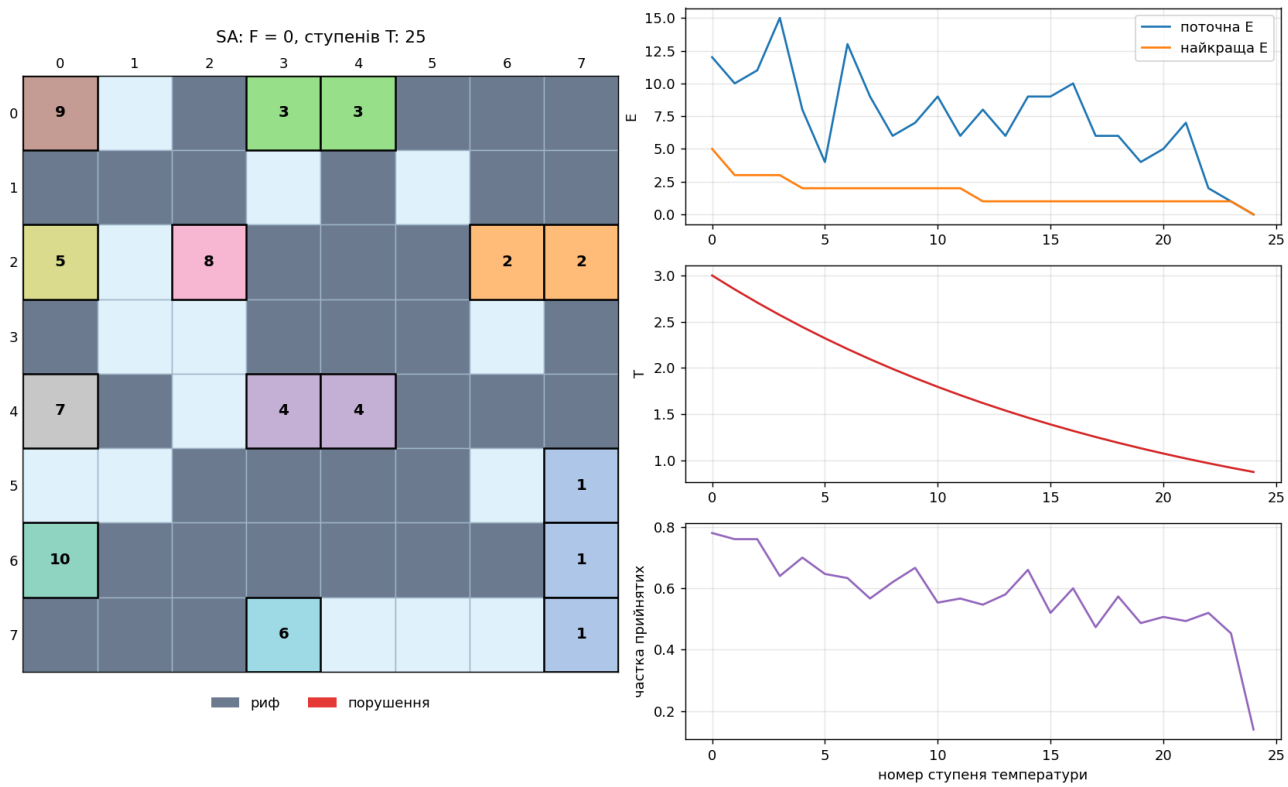


Рис. 4. Розстановка, знайдена SA, та графіки E, T і частки прийнятих переходів.

Завдання 9. Дослідити вплив швидкості охолодження α на результат: щонайменше три значення, для кожного (у варіанті №3 по 30 запусків кожного алгоритму).

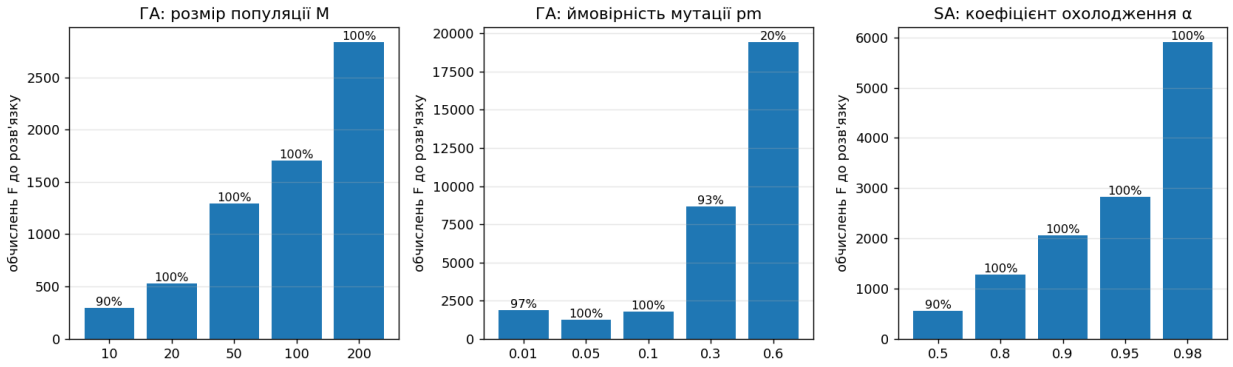


Рис. 5. Середня кількість обчислень F до розв'язку.

Значення	Успішних запусків	Середня к-сть ітерацій	Середнє обчислень F	Середній час, с
0.5	90.0%	4.2	560	0.01
0.8	100.0%	8.9	1271	0.03
0.9	100.0%	14.3	2066	0.05
0.95	100.0%	19.3	2832	0.07
0.98	100.0%	39.9	5915	0.14

```
def research_sa(problem, runs=10):
    return [("alpha", a, summarize([run_sa(problem, alpha=a) for _ in range(runs)]))
            for a in (0.50, 0.80, 0.90, 0.95, 0.98)]
```

Завдання 10. Дослідити вплив швидкості охолодження α на результат: щонайменше три значення, для кожного (у варіанті №3 по 30 запусків кожного алгоритму).

Алгоритм	Успішних запусків	Середнє обчислень F	Середній час, с
Генетичний алгоритм	100.0%	1681	0.05
Імітація відпалу	100.0%	2988	0.07

```
def compare(problem, runs=30):
    ga = summarize([run_ga(problem) for _ in range(runs)])
    sa = summarize([run_sa(problem) for _ in range(runs)])
    return ga, sa
```

Завдання 11. Пояснити, чому для коректного порівняння двох алгоритмів кількість обчислень фітнес-функції є більш показовою метрикою, ніж кількість поколінь або ітерацій. Порівняти поведінку обох алгоритмів на складнішому варіанті (з більшою кількістю рифів, який видає викладач) і зробити висновки.

Кількість обчислень фітнес-функції показовіша, ніж кількість поколінь чи ітерацій: покоління ГА містить ≈ M обчислень, а ступінь температури SA - 150, себто це різні одиниці. Час залежить від мови й комп'ютера, а обчислення F є найдорожчою операцією в обох алгоритмах і від платформи не залежить.

Висновок: У ході виконання лабораторної роботи №3 було розв'язнено задачу розміщення флоту поміж рифів обома методами. Кодування у вигляді хромосоми дозволило автоматично виконувати перші три правила, тоді як перетини й дотики штрафувалися фітнес-функцією до досягнення ідеалу F = 0.

Аналіз ГА показав, що збільшення популяції зменшує кількість поколінь, але збільшує загальну кількість обчислень і оптимум становить 20–50 особи. Найкраща ймовірність мутації лежить у межах 0.05–0.1, оскільки за високого показника (наприклад, 0.6) пошук стає хаотичним і дає лише 20% успіху. Щодо SA, то повільніше охолодження вимагає більше обчислень, проте підвищує надійність (аби досягти 100% успішних запусків, коефіцієнт α має бути ≥ 0.8).

Під час тестування на варіанті №3 обидва алгоритми знайшли розв’язок у 100% запусків. Однак ГА впорався значно ефективніше: йому знадобилося 1681 обчислення фітнес-функції проти 2988 у SA. Це пояснюється тим, що ГА працює відразу з цілою популяцією і завдяки схрещуванню вміє вдало комбінувати фрагменти хороших розстановок, котрі вже знайшов. Своєю чергою, алгоритм імітації відпалу хоч і простіший у налаштуванні, проте рухається лише однією траєкторією пошуку, через що потребує більше спроб.